Chapter 11 | Exercise solutions with Python

Long Context, Sparse Access, and Sequence Compression

This notebook solves Exercises 5–9 and 11. It traces connections in local and global masks, ranks candidates before computing attention, and pools blocks into summary vectors. It also tests a causal summary boundary and measures what happens when the indexer misses a key piece of evidence. Exercises 1–4, 10, and 12 are in the written solutions.

To run the notebook, open it inside a copy of `transformers-book` and select a Python kernel with the packages in `requirements.txt`. Run the cells in order. The exercises run on CPU and include their saved outputs.

```
from pathlib import Path
import sys

import numpy as np

working_directory = Path.cwd()
for repo_root in (working_directory, *working_directory.parents):
    model_path = (
        repo_root / "examples" / "ch11_long_context" / "model.py"
    )
    if model_path.is_file():
        sys.path.insert(0, str(repo_root))
        break
else:
    raise FileNotFoundError(
        "Open this notebook inside transformers-book."
    )
```

5. Inspect a global mask

Imagine seven tokens at positions 0 through 6. A query at position 0 can use its own key and the key immediately to its right under a radius-one rule. Making position 3 global gives the query another place to look even though that key is three positions away. A **mask** records these permissions. In the Boolean array, row q belongs to query position q and column j belongs to key position j. A true entry lets that query read that key.

In the bidirectional rule, a pair is permitted when the query and key are at most one position apart or either position is global. This opens both row 3 and column 3. Causal mode first builds those same permissions and then keeps only the pairs with key position $j \leq q$. The causal rule applies even to the global query and the global key.

```
from examples.ch11_long_context.model import local_mask

masks = {
    "Bidirectional": local_mask(7, 1, global_positions=(3,)),
    "Causal": local_mask(7, 1, global_positions=(3,), causal=True),
}
expected_keys = {
    "Bidirectional": [[0, 1, 3], list(range(7)), [3, 5, 6]],
    "Causal": [[0], [0, 1, 2, 3], [3, 5, 6]],
}
for name, mask in masks.items():
    print(f"{name} mask (1 = readable)")
    print("Key     0 1 2 3 4 5 6")
    for q_pos, row in enumerate(mask):
        row_bits = " ".join(str(int(bit)) for bit in row)
        print(f"Query {q_pos} {row_bits}")
    for q_pos, expected in zip((0, 3, 6), expected_keys[name]):
        readable = np.flatnonzero(mask[q_pos])
        np.testing.assert_array_equal(readable, expected)
        print(f"Keys for query {q_pos}: {readable.tolist()}")
    print()
```

OUTPUT

```
Bidirectional mask (1 = readable)
Key     0 1 2 3 4 5 6
Query 0 1 1 0 1 0 0 0
Query 1 1 1 1 1 0 0 0
Query 2 0 1 1 1 0 0 0
Query 3 1 1 1 1 1 1 1
Query 4 0 0 0 1 1 1 0
Query 5 0 0 0 1 1 1 1
Query 6 0 0 0 1 0 1 1
Keys for query 0: [0, 1, 3]
Keys for query 3: [0, 1, 2, 3, 4, 5, 6]
Keys for query 6: [3, 5, 6]

Causal mask (1 = readable)
Key     0 1 2 3 4 5 6
Query 0 1 0 0 0 0 0 0
Query 1 1 1 0 0 0 0 0
Query 2 0 1 1 0 0 0 0
Query 3 1 1 1 1 0 0 0
Query 4 0 0 0 1 1 0 0
Query 5 0 0 0 1 1 1 0
Query 6 0 0 0 1 0 1 1
Keys for query 0: [0]
Keys for query 3: [0, 1, 2, 3]
Keys for query 6: [3, 5, 6]
```

For query 0, causality removes keys 1 and 3. Key 1 comes from the local window and key 3 comes from the global column, but both are in the future. For the global query at 3, causality removes keys 4, 5, and 6. The query can still read keys 0 through 3, including its own key. Query 6 keeps keys 3, 5, and 6 because all three are at or before it. Earlier keys 0, 1, 2, and 4 remain outside this query's radius and are not global. Causality removes permissions from the bidirectional grid and never adds a read.

6. Filter before gathering

The largest index score, 9.0, belongs to row 3. The allowed vector marks that row as false. The function first forms the candidate set $\{0, 1, 2\}$, then ranks those candidates by index score. The ranking is $[1, 0, 2]$ because $1.1 > 0.8 > 0.6$. Thus the first two picks are rows 1 and 0. When $k = 5$, only three candidates exist. The function uses all three and adds only row 2.

After gathering those keys and values, main attention computes a fresh score for each selected key. The score is the query–key dot product divided by the square root of the key width. Here the query is $[1]$, the width is one, and the score for each row is just its key coordinate. The index scores determine what to gather. The main scores determine how to mix the gathered values.

For $k = 2$, the selected main scores are $(1, 0)$ and their values are $(20, 10)$ in that order. Softmax exponentiates each score and divides by the sum of the exponentials. The weights and output are

$$w = \frac{(e, 1)}{e + 1} \approx (0.731059, 0.268941), \qquad y = \frac{20e + 10}{e + 1} \approx 17.310586.$$

For $k = 5$, the selected main scores are $(1, 0, 2)$, and the values are $(20, 10, 30)$. The newly admitted key gets score 2, the highest **main** score, even though it ranked last among the allowed candidates. The normalization now includes e^2, so the first two weights change as well.

$$w = \frac{(e, 1, e^2)}{e + 1 + e^2} \approx (0.244728, 0.090031, 0.665241), \qquad y = \frac{20e + 10 + 30e^2}{e + 1 + e^2} \approx 25.752104.$$

The output is a one-coordinate vector because each value row has width one. All positions below use zero-based indices. The weights are reported in **selected order**.

```
from math import e

from examples.ch04_attention.attention import (
    scaled_dot_product_attention,
)
from examples.ch11_long_context.model import selected_attention

gather_query = np.array([1.0])
gather_keys = np.arange(4, dtype=float)[:, None]
gather_values = np.array([[10.0], [20.0], [30.0], [40.0]])
gather_scores = np.array([0.8, 1.1, 0.6, 9.0])
gather_allowed = np.array([True, True, True, False])

gather_expected = {
    2: ([1, 0], np.array([e, 1.0]), (20 * e + 10) / (e + 1)),
    5: ([1, 0, 2], np.array([e, 1.0, e**2]),
        (20 * e + 10 + 30 * e**2) / (e + 1 + e**2)),
}
```

```
gathered = {}
for budget, (indices, exp_scores, hand_output) in gather_expected.items():
    output, selected, weights = selected_attention(
        gather_query, gather_keys, gather_values,
        gather_scores, k=budget, allowed=gather_allowed,
    )
    hand_weights = exp_scores / exp_scores.sum()
    np.testing.assert_array_equal(selected, indices)
    np.testing.assert_allclose(
        weights, hand_weights, rtol=0, atol=1e-12,
    )
    np.testing.assert_allclose(
        output, [hand_output], rtol=0, atol=1e-12,
    )
    gathered[budget] = output, selected, weights
    print(f"k = {budget}")
    print(f"  Selected indices  {selected.tolist()}")
    print(f"  Main scores       {gather_keys[selected, 0].tolist()}")
    print(f"  Main weights      {np.round(weights, 8)}")
    print(f"  Output            {output[0]:.8f}")
```

OUTPUT

```
k = 2
  Selected indices  [1, 0]
  Main scores       [1.0, 0.0]
  Main weights      [0.73105858 0.26894142]
  Output            17.31058579
k = 5
  Selected indices  [1, 0, 2]
  Main scores       [1.0, 0.0, 2.0]
  Main weights      [0.24472847 0.09003057 0.66524096]
  Output            25.75210383
```

With $k = 5$, the gathered attention sees every allowed row. Its output therefore agrees with dense attention with the same mask. The dense weights are indexed in original row order, so gather those weights at the selected indices before comparing. The dense weight for row 3 is zero.

```
masked_out, masked_w = scaled_dot_product_attention(
    gather_query[None, :], gather_keys, gather_values,
    allowed_mask=gather_allowed,
)
output, selected, weights = gathered[5]
np.testing.assert_allclose(output, masked_out[0], rtol=0, atol=1e-12)
np.testing.assert_allclose(
    weights, masked_w[0, selected], rtol=0, atol=1e-12,
)
assert masked_w[0, 3] == 0
print("Dense weights (rows 0–3)", masked_w[0].round(8))
print("Dense output with the same mask", masked_out[0, 0].round(8))
```

OUTPUT

```
Dense weights (rows 0–3) [0.09003057 0.24472847 0.66524096 0.        ]
Dense output with the same mask 25.75210383
```

7. Compare summary windows

Imagine writing one short note for every two tokens. A stride of two determines when a new note is ready. An overlapping note can refer to the preceding block as well as the current one. Its end position still marks the same newest token. In this example the feature values are the integers 1 through 6, and their zero-based positions are 0 through 5.

A gate logit is a score that softmax turns into a weight. All gate logits are zero, so in a window of m positions each exponentiated score is $e^0 = 1$ and each weight is $1/m$. With one feature per token, each summary is just the mean of the values in its window.

End position	Positions without overlap	Mean	Positions with overlap	Mean
1	0, 1	$(1 + 2)/2 = 1.5$	0, 1	$(1 + 2)/2 = 1.5$
3	2, 3	$(3 + 4)/2 = 3.5$	0, 1, 2, 3	$(1 + 2 + 3 + 4)/4 = 2.5$
5	4, 5	$(5 + 6)/2 = 5.5$	2, 3, 4, 5	$(3 + 4 + 5 + 6)/4 = 4.5$

The first window ends at position 1 in either mode. It contains two tokens because there is no preceding block at the start. The next two overlapping windows contain four tokens each. The first window's weights are therefore 1/2 per position, while each later overlapping window uses 1/4 per position.

```
from examples.ch11_long_context.model import block_summaries

window_features = np.arange(1, 7, dtype=np.float64)[:, None]
window_logits = np.zeros_like(window_features)
window_cases = (
    (False, [1.5, 3.5, 5.5]),
    (True, [1.5, 2.5, 4.5]),
)

for overlap, expected in window_cases:
    rows, ends = block_summaries(
        window_features, window_logits, stride=2, overlap=overlap
    )
    np.testing.assert_array_equal(ends, [1, 3, 5])
    np.testing.assert_allclose(
        rows[:, 0], expected, atol=1e-12, rtol=0
    )
    print(f"Overlap {overlap}")
    print("  End positions", ends.tolist())
    print("  Summary values", rows[:, 0].tolist())
```

OUTPUT

```
Overlap False
  End positions [1, 3, 5]
  Summary values [1.5, 3.5, 5.5]
Overlap True
  End positions [1, 3, 5]
  Summary values [1.5, 2.5, 4.5]
```

The output has three rows and one feature per row in both modes. The positions $[1, 3, 5]$ record the end of each window. Every completed stride block produces one summary, so the overlapping windows change the values and the source positions without increasing the number of summaries. An unfinished final stride block would produce no row.

8. Give coordinates different gates

Now each token has two features. The first coordinate reads $(2, 0, 6)$ across the three positions. The second reads $(0, 4, 2)$. With stride three there is one completed window, ending at position 2. Each coordinate forms its own weighted sum over that same window.

Softmax exponentiates the logit in each row and divides by the sum of exponentials in that coordinate's column. The first gate column contains the logarithms of $(1, 2, 1)$, so exponentiating recovers those numbers. Their sum is 4, and the weights are $(1/4, 2/4, 1/4)$. The first summary coordinate is

$$\frac{1}{4}(2) + \frac{2}{4}(0) + \frac{1}{4}(6) = 2.$$

The second gate column contains the logarithms of $(3, 1, 2)$. Its exponential sum is 6, giving weights $(3/6, 1/6, 2/6)$. The second coordinate is

$$\frac{3}{6}(0) + \frac{1}{6}(4) + \frac{2}{6}(2) = \frac{8}{6} = \frac{4}{3}.$$

The zero feature at position 1 receives half of the weight in the first coordinate. The zero at position 0 receives half in the second coordinate. Each contributes zero to its sum while still using some of the weight. The weights are determined by the gate logits, so a feature value of zero does not remove its position from the softmax.

```
gate_features = np.array([[2., 0.], [0., 4.], [6., 2.]])
gate_numbers = np.array([[1., 3.], [2., 1.], [1., 2.]])
feature_logits = np.log(gate_numbers)

gate_rows, gate_ends = block_summaries(
    gate_features, feature_logits, stride=3
)
gate_weights = gate_numbers / gate_numbers.sum(axis=0)
gate_hand = (gate_weights * gate_features).sum(axis=0)

np.testing.assert_array_equal(gate_ends, [2])
np.testing.assert_allclose(
    gate_weights.sum(axis=0), [1., 1.], atol=1e-12, rtol=0
)
np.testing.assert_allclose(
    gate_rows, [[2., 4 / 3]], atol=1e-12, rtol=0
)
np.testing.assert_allclose(
    gate_rows[0], gate_hand, atol=1e-12, rtol=0
)

with np.printoptions(precision=8):
    print("Weights (rows are positions, columns are features)")
    print(gate_weights)
    print("Hand result", gate_hand)
    print("Helper result", gate_rows[0])
print("End positions", gate_ends.tolist())
```

```
OUTPUT
Weights (rows are positions, columns are features)
[[0.25       0.5       ]
 [0.5        0.16666667]
 [0.25       0.33333333]]
Hand result [2.         1.33333333]
Helper result [2.         1.33333333]
End positions [2]
```

The helper transposes the gate array before softmax because its softmax normalizes the last axis. That puts positions on the axis being normalized. It transposes the weights back to match the original feature array, multiplies entries coordinate by coordinate, and sums over positions. The comparison uses float64 and an absolute tolerance of 10^{-12} with zero relative tolerance.

9. Check the causal boundary (optional)

At query position 7, the decoder has seen the values 1 through 8. The third stride block starts at position 6, so the decoder can already see its first two values, 7 and 8. Its last value is at position 8, which comes after the query. To decide whether a query can read an entire summary, check the newest source position in that summary.

We can build all three summaries on the completed array and then ask which would be available to the earlier query. A zero gate array gives equal weights over each three-token window. Changing the value at position 8 from 9 to 90 changes only the third window. Its mean increases by $(90 - 9)/3 = 27$.

Source positions	End position	Original mean	New mean	Visible at query 7?
0, 1, 2	2	$(1 + 2 + 3)/3 = 2$	2	Yes
3, 4, 5	5	$(4 + 5 + 6)/3 = 5$	5	Yes
6, 7, 8	8	$(7 + 8 + 9)/3 = 8$	$(7 + 8 + 90)/3 = 35$	No

Causal attention permits a read when the summary's end is at or before the query. Comparing the end positions to 7 gives the same mask after the edit because the source positions of each window are unchanged.

```
boundary_features = np.arange(1, 10, dtype=np.float64)[:, None]
boundary_logits = np.zeros_like(boundary_features)
edited_features = boundary_features.copy()
edited_features[-1, 0] = 90.
query_position = 7

old_summaries, old_ends = block_summaries(
    boundary_features, boundary_logits, stride=3
)
new_summaries, new_ends = block_summaries(
    edited_features, boundary_logits, stride=3
)
visible_before = old_ends <= query_position
visible_after = new_ends <= query_position
np.testing.assert_array_equal(old_ends, [2, 5, 8])
np.testing.assert_array_equal(new_ends, old_ends)
np.testing.assert_array_equal(visible_before, [True, True, False])
np.testing.assert_array_equal(visible_after, visible_before)

print("End positions", new_ends.tolist())
print("Visible at position 7", visible_after.tolist())
with np.printoptions(precision=6):
    print("Original means", old_summaries[:, 0])
    print("Edited means", new_summaries[:, 0])
```

OUTPUT

```
End positions [2, 5, 8]
Visible at position 7 [True, True, False]
Original means [2. 5. 8.]
Edited means [ 2.  5. 35.]
```

For these nonoverlapping windows of width three, the start position is the end minus two. If we test starts instead, every start passes because 0, 3, and 6 are all at or before position 7. That admits the third summary even though it includes the unseen value at position 8.

```
np.testing.assert_allclose(
    old_summaries[:, 0], [2., 5., 8.], atol=1e-12, rtol=0
)
np.testing.assert_allclose(
    new_summaries[:, 0], [2., 5., 35.], atol=1e-12, rtol=0
)
np.testing.assert_allclose(
    old_summaries[visible_before], new_summaries[visible_after],
    atol=1e-12, rtol=0
)
wrong_mask = old_ends - 2 <= query_position
np.testing.assert_array_equal(wrong_mask, [True, True, True])

print("Mask based on window starts", wrong_mask.tolist())
visible_means = new_summaries[visible_after, 0]
print("Visible means with the correct mask", visible_means)
print("Edited third mean exposed by the wrong mask",
      new_summaries[wrong_mask, 0][-1])
```

```
OUTPUT
Mask based on window starts [True, True, True]
Visible means with the correct mask [2. 5.]
Edited third mean exposed by the wrong mask 35.0
```

The correct mask leaves the visible summary values at $(2, 5)$ in both runs. With the start-based mask the query could see a summary that moves from 8 to 35 in response to a future edit. Checking the end prevents that information leak. At position 8 the third block is complete and its summary can first be read. If a decoder also has a recent raw window, it can read the already-seen positions in the unfinished block through that window while the summary remains unavailable.

11. Measure missed evidence (optional)

Imagine one query looking for a fact that appears only in row 2. Give that row a positive first value coordinate, and set the first coordinate to zero on all four distractors. This coordinate lets us see the effect of omitting the target. Give distractors different second coordinates so changes to their relative mixture are also visible.

Use the fixed width-one query $[1]$ and keys of width one. Use natural logarithms of $(1, 2, 12, 3, 1)$ as the key coordinates. The main scores equal those coordinates. Full attention gives weights $(1, 2, 12, 3, 1)/19$ and output $(120/19, 18/19)$ with the values below. It assigns the largest weight to the target. We keep the query, keys, values, and available positions fixed for the entire sweep.

```
sweep_query = np.array([1.0])
sweep_keys = np.log([1.0, 2.0, 12.0, 3.0, 1.0])[:, None]
sweep_values = np.array([
    [0.0, 1.0], [0.0, 2.0], [10.0, 0.0],
    [0.0, 4.0], [0.0, 1.0],
])
target_row = 2
dense_out, dense_w = scaled_dot_product_attention(
    sweep_query[None, :], sweep_keys, sweep_values,
)
dense_out = dense_out[0]
np.testing.assert_allclose(
    dense_out, np.array([120, 18]) / 19, rtol=0, atol=1e-12,
)
print("Dense weights (rows 0–4)", dense_w[0].round(6))
print("Dense output", dense_out.round(6))
```

OUTPUT

```
Dense weights (rows 0–4) [0.052632 0.105263 0.631579 0.157895 0.052632]
Dense output [6.315789 0.947368]
```

The separate indexer gives distractors at positions 0, 1, 3, and 4 fixed scores of 0.9, 0.7, 0.4, and 0.2. Sweep the target index score from 0.1 to 1.0 and try $k = 1, 2, 3, 5$. With a score of 0.4, row 2 ties row 3 and wins because its index comes earlier. The target is then ranked third and fits at $k = 3$. With a score of 0.7 , it ties row 1 and loses that tie. A budget of two still misses the target. The main scores never change.

For every pair of target score and k, record the gathered positions and whether row 2 was among them. Compare the two-coordinate outputs using their Euclidean distance

$$\| y_{selected} - y_{dense} \|_2 = \sqrt{(y_1 - y_{dense,1})^2 + (y_2 - y_{dense,2})^2}.$$

This is a distance in output space. We also print the first coordinate so we can distinguish loss of the target from a change in weighting when the target stays selected.

```
base_index_scores = np.array([0.9, 0.7, 0.0, 0.4, 0.2])
target_scores = (0.1, 0.4, 0.55, 0.7, 0.8, 1.0)
sweep_budgets = (1, 2, 3, 5)
sweep_rows = []
for target_score in target_scores:
    scores = base_index_scores.copy()
    scores[target_row] = target_score
    for budget in sweep_budgets:
        output, selected, _ = selected_attention(
            sweep_query, sweep_keys, sweep_values, scores, k=budget,
        )
        hit = target_row in selected
        distance = np.linalg.norm(output - dense_out)
        sweep_rows.append((
            target_score, budget, selected.tolist(),
            hit, output[0], distance,
        ))
        if not hit:
            assert output[0] == 0
        if budget == len(scores):
            np.testing.assert_allclose(
                output, dense_out, rtol=0, atol=1e-12,
            )
```

```
print("Score  k   Selected indices   Hit   First coord.  Distance")
for score, budget, indices, hit, first, distance in sweep_rows:
    selected_text = ",".join(map(str, indices))
    answer = "yes" if hit else "no"
    print(f"{score:>5.2f}  {budget}   {selected_text:<17} "
          f"{answer:>3}   {first:>11.6f}  {distance:>8.6f}")
```

```
OUTPUT
Score  k   Selected indices   Hit   First coord.  Distance
 0.10  1   0                  no      0.000000  6.316009
 0.10  2   0,1                no      0.000000  6.356618
 0.10  3   0,1,3              no      0.000000  6.591363
 0.10  5   0,1,3,4,2         yes      6.315789  0.000000
 0.40  1   0                  no      0.000000  6.316009
 0.40  2   0,1                no      0.000000  6.356618
 0.40  3   0,1,2             yes      8.000000  1.792653
 0.40  5   0,1,2,3,4         yes      6.315789  0.000000
 0.55  1   0                  no      0.000000  6.316009
 0.55  2   0,1                no      0.000000  6.356618
 0.55  3   0,1,2             yes      8.000000  1.792653
 0.55  5   0,1,2,3,4         yes      6.315789  0.000000
 0.70  1   0                  no      0.000000  6.316009
 0.70  2   0,1                no      0.000000  6.356618
 0.70  3   0,1,2             yes      8.000000  1.792653
 0.70  5   0,1,2,3,4         yes      6.315789  0.000000
 0.80  1   0                  no      0.000000  6.316009
 0.80  2   0,2               yes      9.230769  3.042167
 0.80  3   0,2,1             yes      8.000000  1.792653
 0.80  5   0,2,1,3,4         yes      6.315789  0.000000
 1.00  1   2                  yes     10.000000  3.804065
 1.00  2   2,0                yes      9.230769  3.042167
 1.00  3   2,0,1              yes      8.000000  1.792653
 1.00  5   2,0,1,3,4         yes      6.315789  0.000000
```

A miss erases the target signal in the first coordinate. For example, with a target score of 0.1 and $k = 3$, that coordinate is zero while dense attention gives $120/19 \approx 6.316$. When the target score rises to 1.0 at $k = 1$, the target is the only retained value. Its first coordinate is now 10. This run finds the target but still differs from dense attention, which gives some weight to the distractors. Keeping the target at $k = 3$ with score 1.0 retains two distractors too. The first coordinate becomes $120/(12 + 1 + 2) = 8$. At score 0.1 , raising k from one to three admits only distractors. The first coordinate stays zero and the second moves farther from the dense reference, so the distance grows in this case.

Two distinct mechanisms contribute to the output error. An omitted row contributes nothing. Even when the target is retained, softmax renormalizes across the rows that remain. With $k = 5$, all five rows contribute to both sums in softmax attention. Changing their order leaves the mathematical output unchanged and the NumPy results agree within the tolerance. This controlled fixture measures how ranking changes one attention output. A trained model's answer quality requires a separate test on labeled tasks.